DHAT Boilerplate Migration & Architecture SOP

Django + HTMX + Alpine.js + Tailwind CSS v4 Professional Migration Guide

---

Django + HTMX + Alpine.js + Tailwind CSS v4 Professional Migration Guide

---

■ Executive Summary

This document serves as the official standard operating procedure (SOP) for migrating existing and legacy Django web projects to the modern **DHAT (Django + HTMX + Alpine.js + Tailwind CSS v4)** production-grade architecture.

By applying this guide, your applications gain:

1. **50% Template Reduction**: Eliminating duplicate `*_full.html` templates via dynamic base inheritance.

2. **Zero-Byte-Hack Toast System**: Clean Out-Of-Band (OOB) notifications adhering strictly to top-right placement rules (`fixed top-20 right-6 z-99999`).

3. **Global UX Resilience**: Top loading progress bar, automatic CSRF token injection, and 401/403 session expiration redirects.

4. **12-Factor Production Ready**: Safe `.env` configuration and conflict-free Tailwind CSS v4 asset pipeline.

5. **100% Automated Test Coverage**: Standalone `test.py` test suite for continuous verification.

■ Step-by-Step Migration Checklist

1. Dynamic Base Template Strategy (Single-Template Pattern)

A. Create `templates/base_partial.html`

Create a minimal container with no enclosing HTML document wrapper:

```
{% block content %}{% endblock %}
```

B. Create `config/context_processors.py`

```
def base_template(request):
    """
    Returns 'base_partial.html' for HTMX requests and 'base.html' for direct page loads.
    Eliminates the need for duplicate *_full.html files.
    """
    is_htmx = getattr(request, 'htmx', False)
    return {
        'base_template': 'base_partial.html' if is_htmx else 'base.html',
        'is_htmx': bool(is_htmx),
    }
```

C. Register in `config/settings.py`

Add to `TEMPLATES[0]['OPTIONS']['context_processors']`:

```
'config.context_processors.base_template',
```

D. Update Page Templates & Delete `*_full.html`

In every template under `templates/pages/*.html`, wrap content with:

```
{% extends base_template %}

{% block content %}
<div class="w-full max-w-full space-y-8 p-4 sm:p-6 lg:p-8">
    <!-- Page Content -->
</div>
{% endblock %}
```

Then delete all redundant full templates:

```
rm templates/pages/*_full.html
```

E. Simplify Django Views

Remove branching logic from `views.py`:

```
# ■ Old Redundant Pattern:
if request.htmx:
    return render(request, 'pages/dashboard.html')
return render(request, 'pages/dashboard_full.html')

# ■ New Single-Template Pattern:
return render(request, 'pages/dashboard.html')
```

---

2. Standardized HTMX OOB Toasts & Responses

A. Create `config/htmx_utils.py`

```
from django.http import HttpResponse
from django.shortcuts import render
from django.template.loader import render_to_string

def htmx_redirect(url: str) -> HttpResponse:
    response = HttpResponse()
    response['HX-Redirect'] = url
    return response

def render_htmx(request, template_name: str, context: dict = None, toast_success: str = None,
                toast_error: str = None, form_errors=None, extra_oob: list = None):
    context = context or {}
    if toast_success: context['success'] = toast_success
    if toast_error: context['error'] = toast_error
    if form_errors: context['form_errors'] = form_errors

    response = render(request, template_name, context)

    if getattr(request, 'htmx', False):
        oob_items = []
        if extra_oob:
            oob_items.extend(extra_oob if isinstance(extra_oob, (list, tuple)) else [extra_oob])
        if toast_success or toast_error or form_errors:
            toast_html = render_to_string('partials/toast.html', context, request=request)
            oob_items.append(toast_html)
        if oob_items:
            response.content += "".join(oob_items).encode('utf-8')
    return response

class HtmxMessagesMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        response = self.get_response(request)
        if getattr(request, 'htmx', False) and response.status_code == 200:
            content_type = response.get('Content-Type', '')
            if 'text/html' in content_type and not response.has_header('HX-Redirect'):
                from django.contrib.messages import get_messages
                messages = get_messages(request)
                if messages:
                    toast_html = render_to_string('partials/toast.html', {'messages': messages}, request=request)
                    response.content += toast_html.encode('utf-8')
        return response
```

B. Register Middleware in `config/settings.py`

Add to `MIDDLEWARE` after `MessageMiddleware` and `HtmxMiddleware`:

```
'config.htmx_utils.HtmxMessagesMiddleware',
```

---

3. Global UX & Base Layout (`templates/base.html`)

Add to `templates/base.html`:

1. **Top Loading Progress Bar**:

```html
<div id="loading-bar" class="fixed top-0 left-0 right-0 h-1 bg-gradient-to-r from-brand-500 via-indigo-500 to-purple-600 z-[999999] opacity-0 transition-opacity duration-300 pointer-events-none"></div>
```

2. **Dynamic Modal Container**:

```html
<div id="modal-container"></div>
```

3. **Global HTMX Resilience Script**:

```html
<script>
document.addEventListener('DOMContentLoaded', function() {
// Auto CSRF Injection
document.body.addEventListener('htmx:configRequest', function (evt) {
const cookie = document.cookie.split('; ').find(row => row.startsWith('csrftoken='));
if (cookie) evt.detail.headers['X-CSRFToken'] = decodeURIComponent(cookie.split('=')[1]);
});
// Loading Bar Progress
const bar = document.getElementById('loading-bar');
document.body.addEventListener('htmx:beforeRequest', () => bar && bar.classList.remove('opacity-0'));
document.body.addEventListener('htmx:afterRequest', () => bar && bar.classList.add('opacity-0'));
// Session Timeout Redirect
document.body.addEventListener('htmx:responseError', function (evt) {
if (evt.detail.xhr && (evt.detail.xhr.status === 401 || evt.detail.xhr.status === 403)) {
window.location.href = '/login/?session_expired=1';
}
});
});
</script>
```

---

4. Tailwind CSS v4 & 12-Factor Setup

1. **Relocate Tailwind CLI Input File**:

Move `static/css/input.css` to `assets/css/input.css` to avoid the `django_tailwind_cli.W001` warning and prevent WhiteNoise `CompressedManifestStaticFilesStorage` build crashes.

2. **Configure in `settings.py`**:

```python
TAILWIND_CLI_SRC_CSS = 'assets/css/input.css'
TAILWIND_CLI_DIST_CSS = 'css/style.css'
```

```

3. **Environment Variables**:

Use `django-environ` to read from `.env`:

```python

import environ

env = environ.Env()

environ.Env.read_env(BASE_DIR / '.env')

SECRET_KEY = env('SECRET_KEY')

DEBUG = env.bool('DEBUG', default=False)

DATABASES = {'default': env.db('DATABASE_URL', default=f"sqlite:///{BASE_DIR / 'db.sqlite3'}")}

```

4. **Health Check Endpoint**:

Add `/health/` in `config/urls.py` and `config/views.py`.

---

5. Automated Verification (`test.py`)

Run the test suite to ensure 100% architectural integrity:

```
python3 test.py
```

This tests:

• Direct full-page requests vs HTMX partial swaps

• Standard redirects and `HX-Redirect` headers

• Site settings CRUD and OOB toast rendering

• Production health check `/health/`

■ Ready-to-Use AI Assistant Prompt

Whenever you open a legacy project, simply provide this prompt to your AI assistant:

```
Please migrate this project to our standardized DHAT (Django + HTMX + Alpine.js + Tailwind CSS v4) architecture:
1. Create `templates/base_partial.html` and `config/context_processors.py` with `base_template`.
2. Wrap all `templates/pages/*.html` in `{% extends base_template %}` and delete all redundant `*_full.html` files.
3. Clean up `views.py` to remove duplicate `if request.htmx` template branches.
4. Add `config/htmx_utils.py` and register `HtmxMessagesMiddleware` in `settings.py` for clean OOB toasts.
5. Add the loading progress bar, `#modal-container`, and global CSRF/session handlers to `templates/base.html`.
6. Move Tailwind source to `assets/css/input.css` and configure `.env` with `django-environ`.
7. Add `test.py` and run it to verify all routes and HTMX behaviors pass cleanly.
```
```